

GSOI 2026 暑期常规赛

第三试题解

Ehundategh & tfbz

Gioush 大队

2026 年 7 月

目录

T1 夜奔 (dash)

T2 何以为我 (upline)

T3 浮光掠影 (sunset)

T4 锦鲤 (fish)

吐槽时间

Contents

T1 夜奔 (dash)

T2 何以为我 (upline)

T3 浮光掠影 (sunset)

T4 锦鲤 (fish)

吐槽时间

第一档部分分 20Pts

- 这个时候 $n, m \leq 20$ ，显然可以直接按照 tfbz 经过图腾的顺序模拟。对于目标串中的每个字符，从当前位置开始向前寻找，若走完一整圈仍然没有找到，就说明不存在合法方案。

第一档部分分 20Pts

- ▶ 这个时候 $n, m \leq 20$ ，显然可以直接按照 tfbz 经过图腾的顺序模拟。对于目标串中的每个字符，从当前位置开始向前寻找，若走完一整圈仍然没有找到，就说明不存在合法方案。
- ▶ 由于每个字符至多寻找 n 次，时间复杂度为 $\mathcal{O}(nm)$ 。注意记录字符以后必须继续向前走，不能在同一次经过中重复记录当前位置。

第二档部分分 40Pts

- ▶ 这个时候 $n, m \leq 2000$ ，显然上一档部分分的做法仍然可以通过，每次从上一次停留的图腾继续向后寻找即可。

第二档部分分 40Pts

- ▶ 这个时候 $n, m \leq 2000$ ，显然上一档部分分的做法仍然可以通过，每次从上一次停留的图腾继续向后寻找即可。
- ▶ 若走到第 n 个图腾后仍然没有找到需要的字符，则从第 1 个图腾继续寻找。每个字符最多寻找一整圈，时间复杂度为 $\mathcal{O}(nm)$ 。

第三档部分分 60Pts

- ▶ 若 T 是 P 的子序列，只需要用双指针扫描一遍 P ，依次匹配 T 中的每个字符即可。

第三档部分分 60Pts

- ▶ 若 T 是 P 的子序列，只需要用双指针扫描一遍 P ，依次匹配 T 中的每个字符即可。
- ▶ 每次找到目标字符以后，从下一个位置继续匹配，总时间复杂度为 $\mathcal{O}(n + m)$ 。

第四档部分分 80Pts

- ▶ 若 T 是 $P + P$ 的子序列，那么只需要接着上一档部分分的做法，将 P 扫描两遍即可，时间复杂度仍然为 $\mathcal{O}(n + m)$ 。

第四档部分分 80Pts

- ▶ 若 T 是 $P + P$ 的子序列，那么只需要接着上一档部分的做法，将 P 扫描两遍即可，时间复杂度仍然为 $\mathcal{O}(n + m)$ 。
- ▶ 根据这两档部分的启发，可以发现每次都应该记录当前位置之后最早出现的目标字符。选择更晚的位置只会使当前结束位置后移，不可能让后续匹配更优。

正解 1 100Pts

- ▶ 考虑直接预处理从每个位置出发，下一次遇到每一种字符时会到达的位置，以及中间需要经过多少个图腾。令 $\text{Jump}[i][c]$ 和 $\text{Dis}[i][c]$ 分别保存这两个信息。
- ▶ 注意这里寻找的是位置 i 以后第一次出现的字符，不包括位置 i 本身。这样处理完一个字符以后，下一次一定会继续向前走，不会重复记录当前图腾。
- ▶ 同时预处理每种字符第一次出现的位置 $\text{FPos}[c]$ 。若位置 i 的后面不存在字符 c ，就可以利用 $\text{FPos}[c]$ 直接处理绕回开头的情况。

正解 1 100Pts

- 考虑从 n 到 1 逆序处理。若 S_{i+1} 正好是字符 c ，那么显然有 $\text{Jump}[i][c]=i+1$ ，且只需要经过一个图腾。

正解 1 100Pts

- ▶ 考虑从 n 到 1 逆序处理。若 S_{i+1} 正好是字符 c ，那么显然有 $\text{Jump}[i][c]=i+1$ ，且只需要经过一个图腾。
- ▶ 否则，若从位置 $i+1$ 出发能够找到字符 c ，那么位置 i 的答案与其到达位置相同，而距离需要额外加一。

正解 1 100Pts

- ▶ 考虑从 n 到 1 逆序处理。若 S_{i+1} 正好是字符 c ，那么显然有 $\text{Jump}[i][c]=i+1$ ，且只需要经过一个图腾。
- ▶ 否则，若从位置 $i+1$ 出发能够找到字符 c ，那么位置 i 的答案与其到达位置相同，而距离需要额外加一。
- ▶ 若位置 $i+1$ 以后已经找不到字符 c ，但是该字符在 P 中出现过，那么就需要绕回 $\text{FPos}[c]$ ，此时经过的图腾数量为 $n-i+\text{FPos}[c]$ 。

正解 1 100Pts

- ▶ 考虑从 n 到 1 逆序处理。若 S_{i+1} 正好是字符 c ，那么显然有 $\text{Jump}[i][c]=i+1$ ，且只需要经过一个图腾。
- ▶ 否则，若从位置 $i+1$ 出发能够找到字符 c ，那么位置 i 的答案与其到达位置相同，而距离需要额外加一。
- ▶ 若位置 $i+1$ 以后已经找不到字符 c ，但是该字符在 P 中出现过，那么就需要绕回 $\text{FPos}[c]$ ，此时经过的图腾数量为 $n-i+\text{FPos}[c]$ 。
- ▶ 若字符 c 从未出现，则对应的 Jump 保持为 0。之后遇到这种状态时，直接输出 -1 即可。

正解 1 100Pts

```

1  memset(FPos,0x3f,sizeof(FPos));
2  for (int i=1;i<=n;i++) {
3      FPos[S[i]-'a']=min(i,FPos[S[i]-'a']);
4  }
5  for (int i=n;i>=1;i--) {
6      if (i!=n) {
7          Jump[i][S[i+1]-'a']=i+1;
8          Dis[i][S[i+1]-'a']=1;
9      }
10 }
```

正解 1 100Pts

```

1  for (int i=n;i>=1;i--) {
2      for (int j=0;j<=25;j++) {
3          if (i!=n&&S[i+1]==j+'a') continue;
4          if (i==n||!Jump[i+1][j]) {
5              if (FPos[j]<=n&&i>=FPos[j]) {
6                  Jump[i][j]=FPos[j];
7                  Dis[i][j]=n-(i-FPos[j]);
8              }
9          }
10         else {
11             Jump[i][j]=Jump[i+1][j];
12             Dis[i][j]=Dis[i+1][j]+1;
13         }
14     }
15 }

```

正解 1 100Pts

- 实现时最容易出错的是第一个字符。tfbz 初始位于第 1 个图腾之前，所以需要先经过第 1 个图腾。代码中令 $\text{Now}=1, \text{Ans}=1$ ，再单独判断 S_1 能否记录为 T_1 。

正解 1 100Pts

- ▶ 实现时最容易出错的是第一个字符。tfbz 初始位于第 1 个图腾之前，所以需要先经过第 1 个图腾。代码中令 $\text{Now}=1, \text{Ans}=1$ ，再单独判断 S_1 能否记录为 T_1 。
- ▶ 若 $S_1 \neq T_1$ ，则从位置 1 继续利用预处理结果寻找 T_1 ；之后每次都从当前停留的位置严格向后寻找下一个字符。

正解 1 100Pts

- ▶ 实现时最容易出错的是第一个字符。tfbz 初始位于第 1 个图腾之前，所以需要先经过第 1 个图腾。代码中令 $\text{Now}=1, \text{Ans}=1$ ，再单独判断 S_1 能否记录为 T_1 。
- ▶ 若 $S_1 \neq T_1$ ，则从位置 1 继续利用预处理结果寻找 T_1 ；之后每次都从当前停留的位置严格向后寻找下一个字符。
- ▶ 这样每次转移只需要 $\mathcal{O}(1)$ ，预处理复杂度为 $\mathcal{O}(26n)$ ，总时间复杂度为 $\mathcal{O}(26n + m)$ ，空间复杂度为 $\mathcal{O}(26n)$ 。答案需要使用 long long。

正解 1 100Pts

```
1 int Now=1;
2 long long Ans=1;
3 if (S[1]!=T[1]) {
4     if (!Jump[1][T[1]-'a']) {
5         puts("-1"); return 0;
6     }
7     else {
8         Ans+=Dis[1][T[1]-'a'];
9         Now=Jump[1][T[1]-'a'];
10    }
11 }
```


正解 1 100Pts

```

1  for (int i=2;i<=m;i++) {
2      if (!Jump[Now] [T[i]-'a']) {
3          puts("-1"); return 0;
4      }
5      else {
6          Ans+=Dis[Now] [T[i]-'a'];
7          Now=Jump[Now] [T[i]-'a'];
8      }
9  }

```

正解 2 100Pts

- 也可以对每种字符保存其在 P 中出现的所有位置，这些位置按照编号自然有序。维护当前停留位置 p ，在对应字符的位置序列中二分查找严格大于 p 的第一个位置。

正解 2 100Pts

- ▶ 也可以对每种字符保存其在 P 中出现的所有位置，这些位置按照编号自然有序。维护当前停留位置 p ，在对应字符的位置序列中二分查找严格大于 p 的第一个位置。
- ▶ 若能够找到，就直接移动到该位置；若不存在，则先经过第 $p+1 \sim n$ 个图腾，再从第 1 个图腾开始寻找该字符第一次出现的位置。

正解 2 100Pts

- ▶ 也可以对每种字符保存其在 P 中出现的所有位置，这些位置按照编号自然有序。维护当前停留位置 p ，在对应字符的位置序列中二分查找严格大于 p 的第一个位置。
- ▶ 若能够找到，就直接移动到该位置；若不存在，则先经过第 $p+1 \sim n$ 个图腾，再从第 1 个图腾开始寻找该字符第一次出现的位置。
- ▶ 如果某种字符的位置序列为空，那么显然不存在合法方案。二分时必须使用 `upper_bound`，否则会错误地再次记录当前位置。

正解 2 100Pts

- ▶ 为什么每次选择最早出现的位置一定最优？考虑任意一种合法方案，将其当前选择的位置替换成更早出现的同字符位置，不会增加这一步经过的图腾数量，同时不会使之后的起点后移，所以后续方案仍然合法且不会更劣。
- ▶ 因此逐次选择当前位置以后第一次出现的目标字符即可得到最优解。预处理所有位置的复杂度为 $\mathcal{O}(n)$ ，每次二分复杂度为 $\mathcal{O}(\log n)$ ，总时间复杂度为 $\mathcal{O}(n + m \log n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

正解 2 100Pts

```

1  for (int i=1;i<=n;i++) Pos[s[i]-'a'].push_back(i);
2  for (int i=1;i<=m;i++) {
3      int c=t[i]-'a';
4      if (Pos[c].empty()) {puts("-1");return 0;}
5      vector<int>::iterator It=
6          upper_bound(Pos[c].begin(),Pos[c].end(),Now);
7      if (It==Pos[c].end()) {
8          Ans+=n-Now;
9          Now=0;
10         It=Pos[c].begin();
11     }
12     Ans+=*It-Now;
13     Now=*It;
14 }

```

Contents

T1 夜奔 (dash)

T2 何以为我 (upline)

T3 浮光掠影 (sunset)

T4 锦鲤 (fish)

吐槽时间

第一档部分分 30Pts

题目给定 $n \leq 300$ ，那么可以想到，最大的时间复杂度应为 $\mathcal{O}(n^3)$ 。那我们可以考虑枚举 i, j, k 三重循环，直接判断是否构成经验能力子列即可。时间复杂度 $\mathcal{O}(n^3)$ 。

第二档部分分 60Pts

在题目的 n 的上界变化时，最差时间复杂度不得超过 $\mathcal{O}(n^2)$ ，那么考虑一个经验能力子列的构成，发现其可以拆成两段顺序对，即对于一组 i, j, k ，其实可以拆解为 $a_i < a_j$ 和 $a_j < a_k$ 两段。那么对于一个确定的位置 j ，我们可以求出以其为开头的顺序对个数 f_j 和以其为结尾的顺序对个数 b_j ，两者相乘即为以 j 为中间项的经验能力子列个数。

那么由此，我们直接预处理顺序对个数，枚举 i, j ，若其为顺序对，则将 f_i 和 b_j 分别加一，该步骤复杂度为 $\mathcal{O}(n^2)$ ，最后再枚举每个中间项求出答案即可，该步骤时间复杂度 $\mathcal{O}(n)$ ，总体时间复杂度为 $\mathcal{O}(n^2)$ 。

正解 100Pts

考虑刚刚的解法，瓶颈是统计顺序对个数，根据之前做过的逆序对问题，我们可以套用树状数组解决。例如对于 b_i 的求解，利用权值树状数组，从 1 到 n 遍历，每遍历到一个数 i ，将树状数组 a_i 位加一，那么以其为结尾的顺序对个数直接查找树状数组中 $1 \sim a_i - 1$ 的和即可得到。

同理反向遍历一次，可以求出 f_i ，该过程的时间复杂度为 $\mathcal{O}(n \log n)$ 。最后同样进行枚举中间数的过程即可，时间复杂度为 $\mathcal{O}(n \log n)$ 。答案需要使用 long long 存储。

Contents

T1 夜奔 (dash)

T2 何以为我 (upline)

T3 浮光掠影 (sunset)

T4 锦鲤 (fish)

吐槽时间

第一档部分分 20Pts

- ▶ 此时 $n, m \leq 2000$ 。修改时直接改变原数组，询问时在 $[l, r]$ 上跑一遍最大子段和即可，时间复杂度为 $\mathcal{O}(nm)$ 。

第一档部分分 20Pts

- ▶ 此时 $n, m \leq 2000$ 。修改时直接改变原数组，询问时在 $[l, r]$ 上跑一遍最大子段和即可，时间复杂度为 $\mathcal{O}(nm)$ 。
- ▶ 题目要求选择非空区间，因此当询问区间全部为负数时，答案是其中最大的单个数，而不是 0。

第二档部分分 40Pts

- ▶ 特殊性质保证所有亮度始终非负，那么询问区间中的所有位置都应该被选中，答案就是区间和。

第二档部分分 40Pts

- ▶ 特殊性质保证所有亮度始终非负，那么询问区间中的所有位置都应该被选中，答案就是区间和。
- ▶ 用树状数组维护单点赋值带来的差值，并查询区间和即可，时间复杂度为 $\mathcal{O}((n + m) \log n)$ 。

第三档部分分 80Pts

- ▶ 此时任意时刻负数位置不超过 20。用树状数组维护区间和，再用有序集合维护所有负数位置。

第三档部分分 80Pts

- ▶ 此时任意时刻负数位置不超过 20。用树状数组维护区间和，再用有序集合维护所有负数位置。
- ▶ 一次询问中，负数之间的整段非负区间可以直接压成它们的区间和，于是只剩下至多 41 个数，在压缩后的序列上求最大子段和即可。

正解 1 100Pts

- 考虑线段树上的一个区间，需要维护区间和、最大前缀和、最大后缀和以及最大子段和。合并左右儿子时，跨过中点的候选答案就是左儿子的最大后缀和加右儿子的最大前缀和。

正解 1 100Pts

- ▶ 考虑线段树上的一个区间，需要维护区间和、最大前缀和、最大后缀和以及最大子段和。合并左右儿子时，跨过中点的候选答案就是左儿子的最大后缀和加右儿子的最大前缀和。
- ▶ 单点赋值只需要修改一条根到叶子的路径；区间查询时按照从左到右的顺序合并节点信息。总时间复杂度为 $\mathcal{O}((n+m)\log n)$ ，所有信息均需使用 long long。

正解 1 100Pts

```
1 #define LSon Now<<1
2 #define RSon Now<<1|1
3 struct node{
4     long long Sum,Left,Right,Ans;
5 }Node[MAXN<<2];
6 node Merge(node x,node y){
7     node Ret;
8     Ret.Sum=x.Sum+y.Sum;
9     Ret.Left=max(x.Left,x.Sum+y.Left);
10    Ret.Right=max(y.Right,y.Sum+x.Right);
11    Ret.Ans=max(max(x.Ans,y.Ans),x.Right+y.Left);
12    return Ret;
13 }
14 void Update(int Now){Node[Now]=Merge(Node[LSon],Node[RSon]);}
```

正解 1 100Pts

```

1 void Give(int Now,int Left,int Right,int Pos,long long Val){
2     if(Left==Right){Node[Now]={Val,Val,Val,Val};return;}
3     int Mid=(Left+Right)>>1;
4     if(Pos<=Mid) Give(LSon,Left,Mid,Pos,Val);
5     else Give(RSon,Mid+1,Right,Pos,Val);
6     Update(Now);
7 }
8 node Query(int Now,int Left,int Right,int L,int R){
9     if(L<=Left&&Right<=R) return Node[Now];
10    int Mid=(Left+Right)>>1;
11    if(R<=Mid) return Query(LSon,Left,Mid,L,R);
12    if(L>Mid) return Query(RSon,Mid+1,Right,L,R);
13    return Merge(Query(LSon,Left,Mid,L,R),
14                 Query(RSon,Mid+1,Right,L,R));
15 }

```

正解 2 100Pts

- ▶ 回顾最大子段和的 DP，令 dp_i 表示以第 i 个数结尾的最大子段和，那么有 $dp_i = \max(dp_{i-1} + a_i, a_i)$ ，再对所有 dp_i 取最大值即可。

正解 2 100Pts

- ▶ 回顾最大子段和的 DP，令 dp_i 表示以第 i 个数结尾的最大子段和，那么有 $dp_i = \max(dp_{i-1} + a_i, a_i)$ ，再对所有 dp_i 取最大值即可。
- ▶ 换个角度来看，在依次加入每一个数的过程中，我们只需要维护还没有开始选择、正在选择一段连续区间、已经结束选择三种状态。

正解 2 100Pts

- ▶ 回顾最大子段和的 DP，令 dp_i 表示以第 i 个数结尾的最大子段和，那么有 $dp_i = \max(dp_{i-1} + a_i, a_i)$ ，再对所有 dp_i 取最大值即可。
- ▶ 换个角度来看，在依次加入每一个数的过程中，我们只需要维护还没有开始选择、正在选择一段连续区间、已经结束选择三种状态。
- ▶ 每加入一个数，实际上就是进行一次 DP 状态转移。于是自然可以想到，用矩阵描述这一次转移，并用矩阵乘法合并连续的多次转移。

矩阵构造

- 将正在选择、还没有开始选择、已经结束选择依次编号为 1, 2, 3。令当前数 a 对应的转移矩阵为 $A(a) = (a_{ij})$ ，其中第 i 行第 j 列表示加入当前数以前处于状态 i ，加入以后转移到状态 j 能够取得的最大贡献。

矩阵构造

- ▶ 将正在选择、还没有开始选择、已经结束选择依次编号为 1, 2, 3。令当前数 a 对应的转移矩阵为 $A(a) = (a_{ij})$ ，其中第 i 行第 j 列表示加入当前数以前处于状态 i ，加入以后转移到状态 j 能够取得的最大贡献。
- ▶ 对于当前的数 a ，若原来正在选择，那么可以继续选择，或者选择 a 后结束，所以 $1 \rightarrow 1$ 和 $1 \rightarrow 3$ 的贡献均为 a 。

矩阵构造

- ▶ 将正在选择、还没有开始选择、已经结束选择依次编号为 1, 2, 3。令当前数 a 对应的转移矩阵为 $A(a) = (a_{ij})$, 其中第 i 行第 j 列表示加入当前数以前处于状态 i , 加入以后转移到状态 j 能够取得的最大贡献。
- ▶ 对于当前的数 a , 若原来正在选择, 那么可以继续选择, 或者选择 a 后结束, 所以 $1 \rightarrow 1$ 和 $1 \rightarrow 3$ 的贡献均为 a 。
- ▶ 若还没有开始选择, 可以跳过 a , 也可以从 a 开始选择, 或者只选择 a 就结束, 所以 $2 \rightarrow 2$ 的贡献为 0, $2 \rightarrow 1$ 和 $2 \rightarrow 3$ 的贡献均为 a 。

矩阵构造

- ▶ 若已经结束选择，那么之后的数都不能再选，所以只有 $3 \rightarrow 3$ 合法，贡献为 0。其他没有提到的转移均不合法，令其贡献为负无穷。

矩阵构造

- ▶ 若已经结束选择，那么之后的数都不能再选，所以只有 $3 \rightarrow 3$ 合法，贡献为 0。其他没有提到的转移均不合法，令其贡献为负无穷。
- ▶ 于是，对于一个数 a ，可以得到转移矩阵：

$$A(a) = \begin{pmatrix} a & -\infty & a \\ a & 0 & a \\ -\infty & -\infty & 0 \end{pmatrix}.$$

矩阵构造

- 初始时显然处于第 2 种状态。若询问区间 $[l, r]$ 对应的转移矩阵为 $A_{l,r} = (a_{ij})$ ，那么经过整个区间以后，选择的连续区间可能仍然延伸到右端点，也可能已经结束，所以答案为 $\max(a_{21}, a_{23})$ 。

矩阵构造

- ▶ 初始时显然处于第 2 种状态。若询问区间 $[l, r]$ 对应的转移矩阵为 $A_{l,r} = (a_{ij})$ ，那么经过整个区间以后，选择的连续区间可能仍然延伸到右端点，也可能已经结束，所以答案为 $\max(a_{21}, a_{23})$ 。
- ▶ 注意不能取第 2 行第 2 列，否则就会把一个数都不选择的情况计入答案，这也正好保证了所选区间非空。

矩阵构造

- ▶ 初始时显然处于第 2 种状态。若询问区间 $[l, r]$ 对应的转移矩阵为 $A_{l,r} = (a_{ij})$ ，那么经过整个区间以后，选择的连续区间可能仍然延伸到右端点，也可能已经结束，所以答案为 $\max(a_{21}, a_{23})$ 。
- ▶ 注意不能取第 2 行第 2 列，否则就会把一个数都不选择的情况计入答案，这也正好保证了所选区间非空。
- ▶ DP 转移时需要将贡献相加，并在所有转移中取最大值。若 $C = A \times B$ ，那么本题中的矩阵乘法满足 $c_{ij} = \max_k(a_{ik} + b_{kj})$ 。两个相邻区间的矩阵相乘，就是它们合并后的转移矩阵。

正解 2 100Pts

```
1  const long long INF=400000000000000000011;  
2  class matrix{  
3      long long Num[4][4];  
4  public:  
5      void Init(long long a){for(int i=1;i<=3;i++) for(int j=1;j<=3;j++)  
        Num[i][j]=a;}  
6      matrix(){Init(-INF);}  
7      matrix(long long a){Init(a);}  
8  };
```

正解 2 100Pts

```
1 class matrix{
2     long long Num[4][4];
3 public:
4     matrix operator*(const matrix &t){
5         matrix ans;
6         for(int i=1;i<=3;i++){
7             for(int j=1;j<=3;j++){
8                 for(int k=1;k<=3;k++){
9                     ans.Num[i][j]=max(ans.Num[i][j],
10                        Num[i][k]+t.Num[k][j]);
11                 }
12             }
13         }
14         return ans;
15     }
```

正解 2 100Pts

```
1 void Modify(long long a){  
2     Num[1][1]=Num[1][3]=Num[2][1]=Num[2][3]=a;  
3     Num[2][2]=Num[3][3]=0;  
4 }  
5 auto Back(){return max(Num[2][1],Num[2][3]);}  
6 };
```

- 叶节点直接调用 Modify，查询完成后调用 Back，即可得到对应区间的最大子段和。

正解 2 100Pts

- 在线段树中，每个节点维护对应区间的转移矩阵。若节点维护 $[l, r]$ ，那么其保存 $A(a_l) \times A(a_{l+1}) \times \cdots \times A(a_r)$ 。区间查询时同样必须从左到右合并。

正解 2 100Pts

- ▶ 在线段树中，每个节点维护对应区间的转移矩阵。若节点维护 $[l, r]$ ，那么其保存 $A(a_l) \times A(a_{l+1}) \times \cdots \times A(a_r)$ 。区间查询时同样必须从左到右合并。
- ▶ 每次修改和查询都只会经过线段树上的 $\mathcal{O}(\log n)$ 个节点，矩阵大小为常数，所以总时间复杂度为 $\mathcal{O}((n + m) \log n)$ 。

正解 2 100Pts

- ▶ 在线段树中，每个节点维护对应区间的转移矩阵。若节点维护 $[l, r]$ ，那么其保存 $A(a_l) \times A(a_{l+1}) \times \dots \times A(a_r)$ 。区间查询时同样必须从左到右合并。
- ▶ 每次修改和查询都只会经过线段树上的 $\mathcal{O}(\log n)$ 个节点，矩阵大小为常数，所以总时间复杂度为 $\mathcal{O}((n + m) \log n)$ 。
- ▶ 回顾整个过程，我们实际上是将一段 DP 转移作为线段树维护的信息，并在修改以后重新合并这些转移。这就是动态 DP 的基础。

Contents

T1 夜奔 (dash)

T2 何以为我 (upline)

T3 浮光掠影 (sunset)

T4 锦鲤 (fish)

吐槽时间

第一档部分分 16Pts

- ▶ 这个时候 $n, q \leq 300$, 对每次询问做区间 DP。令 dp_i 表示覆盖到位置 i 所需的最少鱼团数量, 再枚举最后一段的左端点并直接维护区间和与最小值。

第一档部分分 16Pts

- ▶ 这个时候 $n, q \leq 300$, 对每次询问做区间 DP。令 dp_i 表示覆盖到位置 i 所需的最少鱼团数量, 再枚举最后一段的左端点并直接维护区间和与最小值。
- ▶ 直接实现的时间复杂度为 $\mathcal{O}(qn^2)$, 可以通过这一档数据。

第二档部分分 32Pts

- 此时所有 w_i 相等。一个长度为 d 的区间合法，当且仅当 $d \leq k$ 。

第二档部分分 32Pts

- ▶ 此时所有 w_i 相等。一个长度为 d 的区间合法，当且仅当 $d \leq k$ 。
- ▶ 因此长度为 $r - l + 1$ 的询问最少划分成 $\left\lceil \frac{r - l + 1}{k} \right\rceil$ 段，直接计算即可。

性质观察

- 考虑固定一段区间的左端点，并不断向右加入新的鱼。此时区间内的祥瑞值之和只会增大，而最小祥瑞值只会减小或保持不变。

性质观察

- ▶ 再考虑相邻的两个左端点。若 $[i, r]$ 是献瑞鱼团，那么删去第 i 条鱼以后，区间和只会减小，区间最小值只会增大或保持不变，所以 $[i + 1, r]$ 也一定是献瑞鱼团。

性质观察

- ▶ 再考虑相邻的两个左端点。若 $[i, r]$ 是献瑞鱼团，那么删去第 i 条鱼以后，区间和只会减小，区间最小值只会增大或保持不变，所以 $[i + 1, r]$ 也一定是献瑞鱼团。
- ▶ 这说明每个左端点能够到达的最远位置单调不降。于是可以使用双指针维护区间和，同时使用单调队列维护区间最小值，在线性时间内处理出每个位置能够到达的最远位置。

贪心

- ▶ 对于当前还没有被划分的第一条鱼，显然应该让包含它的献瑞鱼团尽可能向右延伸。假设某个最优方案的第一段没有到达能够到达的最远位置，那么我们直接将第一段向右延长。

贪心

- ▶ 对于当前还没有被划分的第一条鱼，显然应该让包含它的献瑞鱼团尽可能向右延伸。假设某个最优方案的第一段没有到达能够到达的最远位置，那么我们直接将第一段向右延长。
- ▶ 延长后的第一段仍然合法，同时剩余部分只会变短，因此需要划分出的献瑞鱼团数量不会增加。不断使用这个过程，就得到每次都取最远位置的贪心做法。

第三档部分分 52Pts

- 这一档只有 30 次询问，所以对每次询问直接进行贪心即可。每次从还没有被划分的第一条鱼出发，找到其能够到达的最远位置，再从下一条鱼继续。

第四档部分分 72Pts

- ▶ 继续使用上一档部分分的贪心。由于这一档保证每次询问的答案不超过 50，所以一次询问至多只会进行 50 次贪心跳转。

第四档部分分 72Pts

- ▶ 继续使用上一档部分分的贪心。由于这一档保证每次询问的答案不超过 50，所以一次询问至多只会进行 50 次贪心跳转。
- ▶ 总时间复杂度为 $\mathcal{O}(n + 50q)$ ，可以通过这一档数据。

正解 100Pts

- 根据上一档部分分的启发，最后只需要加速连续进行贪心的过程。这个过程与树上倍增较为相似，可以预处理从每个位置出发，连续划分 $1, 2, 4, \dots$ 个猹瑞鱼团以后，下一条还没有被划分的鱼的位置。

正解 100Pts

- ▶ 根据上一档部分分的启发，最后只需要加速连续进行贪心的过程。这个过程与树上倍增较为相似，可以预处理从每个位置出发，连续划分 $1, 2, 4, \dots$ 个猹瑞鱼团以后，下一条还没有被划分的鱼的位置。
- ▶ 对于一次询问，从大到小枚举倍增的层数。若连续划分这些猹瑞鱼团以后，仍然没有越过询问的右端点，那么就将这些鱼团全部计入答案，并从新的位置继续。

正解 100Pts

- 在枚举每一层时，我们始终保证已经计入的鱼团都完全包含在询问区间中，因此不会遗漏已经可以确定的完整鱼团。按照层数从大到小枚举以后，得到的就是右端点之前能够完整放下的最多鱼团数量。

正解 100Pts

- ▶ 在枚举每一层时，我们始终保证已经计入的鱼团都完全包含在询问区间中，因此不会遗漏已经可以确定的完整鱼团。按照层数从大到小枚举以后，得到的就是右端点之前能够完整放下的最多鱼团数量。
- ▶ 此时还剩下最后一段没有被计入，而根据前面的贪心证明，这一段直接取到询问右端点即可。于是答案再加一。预处理复杂度为 $\mathcal{O}(n \log n)$ ，单次询问复杂度为 $\mathcal{O}(\log n)$ 。

Contents

T1 夜奔 (dash)

T2 何以为我 (upline)

T3 浮光掠影 (sunset)

T4 锦鲤 (fish)

吐槽时间